

Individual Proposal

MoneyPrinterTurbo Study

Basmala Shtayeh

AI and Automation Stack Engineering

Supervisor: Nader Hantash

Grids AI Studio, AI Team and Internship

Table of Contents

1. My info 3
2. MoneyPrinterTurbo and its pipeline, in my own words 3
3. Code Map 3
4. What I ran: what worked, errors, and keys needed 4
4.1 Setup and API keys 4
4.2 Errors I hit 4
4.3 First video (English) 5
4.4 Arabic test and my fix 5
5. Strengths 6
6. Gaps 6
7. My vision for the Grids AI Studio MVP 7
8. The module I'll own 7
9. 2 week plan 7
10. Risks and open questions 8

1. My info

Item	Answer
Name	Basmala Shtayeh
Track	AI and Automation Stack Engineering
Technologies I know	Languages: Python, JavaScript, TypeScript, Java, C++, PHP, SQL. AI and ML: PyTorch, scikit-learn, NumPy, Pandas, neural networks, generative models (CVAE, GAN), transfer learning (ResNet50, MobileNetV2), NLP, embeddings, Azure AI Foundry. Data: clustering, PCA, Apriori, CRISP DM, ETL, star schema design, SQLAlchemy. Web and backend: React, Vite, MUI, FastAPI, Node.js, Prisma, PHP. Databases: MySQL, Elasticsearch, Firebase. Tools: Docker, Git and GitHub, Azure DevOps, Azure Pipelines, Playwright, Agile and Scrum.
Preferred role	NLP and Scene Intelligence, with interest in AI research and experiments

2. MoneyPrinterTurbo and its pipeline, in my own words

MoneyPrinterTurbo is a free, open source Python tool that turns a topic into a short video automatically. It writes the script, records the voice, finds stock footage, and adds subtitles and music, and Grids AI Studio uses it as a reference for building its own product.

The process of generating a video revolves around six main steps. Before these steps happen, we enter a prompt: we write the title, and we have the option of writing the script text ourselves or not, and of adjusting other settings. If we don't write the text, we can use AI generation (LLM models) to fill it in for us. Then we click Generate. Once Generate is clicked, the API doesn't wait for the video to be finished; instead, it creates a task and returns its ID so we can track the process, while the generation runs in the background.

In the background, six main steps happen. First, it generates the script (if we didn't write one), and from the script it takes about five short English search phrases that it uses to search the Pixabay library. After it has the script and the keywords, it saves them together in a JSON file. After that, it goes to the voice: it turns the text into sound and does a mapping, so while it speaks the text out loud, it records which word is said at which second. After that comes the subtitles: it takes this time mapping and creates a subtitle file (subtitle.srt). After the subtitles, the clips part happens: it searches and downloads clips until there is enough footage to cover the same length as the voice. Finally, it renders the video in two steps: first, it cuts the clips to length, so no single clip takes up the entire video, and joins them; second, it lays the voice, subtitles and background music over them.

One weakness I noticed is that it only picks videos based on the five English search words, just to fill the voice length. So, by default, what's spoken doesn't have to relate to what's being shown; it's pure luck.

3. Code Map

I built this map by following a video request from the start until the final video file.

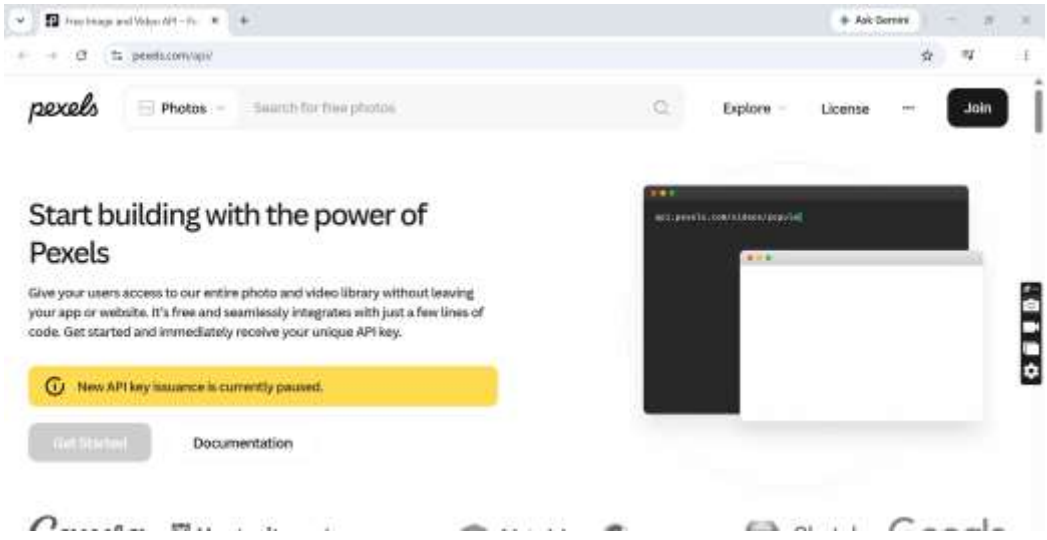
File	What it does
main.py	Starts the API server.
webui/Main.py	The front end: the website where I set things up and clicked Generate.
cli.py	The terminal version, which I used for my Arabic tests.
app/controllers/v1/video.py	Receives a make a video request through the API, creates a task, and returns the task ID.
app/services/task.py	Where the six stages happen, in order.

File	What it does
app/services/llm.py	Writes the script and picks the five English search phrases.
app/models/llm_provider.py	The list of AI companies I can choose from.
app/services/material.py	Searches the footage library (Pixabay or Pexels) with the five phrases and downloads the clips.
app/services/voice.py	Makes the voice and records when every word is said.
app/services/subtitle.py	The Whisper way: makes subtitles by listening to the audio. It's off by default.
app/services/video.py	Joins the clips and adds the voice, subtitles and music. The Arabic fix was added here.
app/config/config.py	Loads the settings and keys.
app/services/state.py	Tracks the task state: running, done or failed.

4. What I ran: what worked, errors, and keys needed

4.1 Setup and API keys

I ran MoneyPrinterTurbo v1.3.7 on my Windows 11 laptop, which has 4 CPU cores, 7.7 GB of RAM and no NVIDIA GPU. That is the minimum the README asks for. I installed it with uv using uv sync --frozen. It downloaded Python 3.11 and 120 packages in about 2 minutes, with no errors, and FFmpeg came bundled, so I didn't have to install it myself. For the keys, the default footage source is Pexels, but Pexels had paused new API keys, so I used Pixabay instead. For the AI model I used Gemini's free tier, and the connection test worked with gemini-3.5-flash. One thing I noticed is that the WebUI saves a key under whichever provider is selected at that moment, so twice my key went into the wrong field: Pexels instead of Pixabay, and Kimi instead of Gemini.



4.2 Errors I hit

Most of my problems came from outside services. When I typed the model name as "flash", Gemini returned a 404 because it isn't a real model name, and some other names were retired or too busy. The default free voice, Edge TTS, was refused by Microsoft with a 403 error, even with the newest version of the library, so I switched to Gemini TTS, which uses the same Gemini key. When Gemini was busy (503), the project retried 5 times in about 17 seconds without waiting. Later I hit the free daily limit (429). Google said to retry in 13 hours, but the project still retried 5 times in about 1 second.





4.3 First video (English)

My first video was about "Why the ocean matters", with Pixabay footage and Gemini's voice. It came out as a 34 second vertical video (1080×1920). It took about 19 minutes to make: about 1 minute for the script and voice, 14 minutes to download and join the clips, and 5 minutes for the final render. The clips were general ocean footage, not matched to what was being said. One subtitle was 26 words long and stayed on screen for almost 10 seconds.

first trial

second trial

4.4 Arabic test and my fix

Arabic isn't in the WebUI's language list, but when I wrote the topic in Arabic, the script came out in Arabic. From the terminal, I could also ask for "ar-SA" or "Palestinian Arabic dialect", and the dialect request really gave a dialect script. The voice in my full Arabic video was fine, but the subtitles came out as boxes because the default font has no Arabic letters. With an Arabic font (Tahoma) the letters appeared, but flipped and not connected. To fix it, I added two libraries, arabic-reshaper (connects the letters) and python-bidi (right to left order), and a small change in video.py, with Claude's help. After the fix, the test frame showed correct Arabic, and the 200 existing tests plus 7 new ones passed.

Arabic trial



5. Strengths

- **Switching providers is just a setting.** I experienced this myself. When Pexels stopped giving out new API keys, I switched to Pixabay, and when the free Edge voice was blocked, I switched to Gemini TTS. Later, when I hit the free Gemini limit, I changed the model from the settings. None of this needed any code changes.
- **Subtitle timing without an extra AI.** It doesn't need an external AI to read the text and work out the timing. The system does it by itself: while the voice speaks, it records when each word is said and uses that for the subtitles.
- **Running only part of the pipeline.** I used this myself when testing Arabic dialects. Adding --stop-at script made it generate only the script and stop, which took about 4 seconds instead of about 20 minutes for a full video. This makes it fast to test one step on its own.

6. Gaps

- **Arabic is not really supported.** Arabic isn't one of the options in the language dropdown, so I had to try it from the terminal (CLI) to see how it works. The script came out fine in both formal Arabic and Palestinian dialect, but this isn't easy for normal users. When I made a full Arabic video, the subtitles showed as boxes, and with an Arabic font the letters were flipped and not connected. To fix this, I added two libraries, arabic-reshaper and python-bidi, and a small change in the code.
- **It was slow on my laptop.** A 34 second video took about 19 minutes on my laptop. Most of that time went to downloading and joining the clips.
- **I ran out of free quota without noticing.** When Gemini was busy, the project retried 5 times very fast without waiting, and it even kept retrying after Google said to wait 13 hours. This used up my free daily limit. The app doesn't show how many requests or credits I have left, so I didn't know until it stopped working.

7. My vision for the Grids AI Studio MVP

I think the MVP should cover what the brief asks for (Audio to Video, Script to Video, Scene Intelligence, and Captions and Rendering), but improve on the problems I saw in MoneyPrinterTurbo:

- **Arabic first.** Arabic and its dialects should be real options in the language list, Arabic fonts should be built in, and captions should never break. In MoneyPrinterTurbo, Arabic wasn't in the list, and the subtitles showed boxes and then flipped letters.
- **Visuals that match what's said.** Keywords should be made for each sentence, not just 5 for the whole video. In my ocean video, the clips were general ocean footage that didn't match what was being said.
- **Show limits and costs.** Users should see how many credits or requests they have left. I ran out of my free Gemini limit without knowing.
- **Backup providers.** If a footage or voice service stops working, the product should switch to another one automatically. Pexels paused new keys and Edge TTS was blocked while I was testing.
- **Faster, if possible.** My 34 second video took about 19 minutes on my laptop, so it would help to make videos faster, for example by doing the heavy work on servers.

8. The module I'll own

I want to own NLP and Scene Intelligence. I chose it because it's one of the top priorities for the MVP and the area I can learn the most from. It also fixes the main weakness I found myself: in MoneyPrinterTurbo, the clips don't match what is being said.

The module reads the script, or the transcript of an uploaded recording, and splits it into scenes using the sentences and the voice timings. Instead of 5 keywords for the whole video, it writes one search phrase per sentence, so each clip is chosen for what is being said at that moment. The phrase is always in English, even when the script is in Arabic or a dialect. The output is a scene JSON file that the next part of the platform uses to find or generate a visual for each scene. In Roomify at ASAL, I built the AI Room Fit Assistant, which also matched AI output to real items from a catalog, so this kind of matching is close to what I have done before.

How it will be measured: on a test set of 10 scripts (5 English and 5 Arabic), I will check how many scenes get a clip that matches what is said, and compare that with MoneyPrinterTurbo's 5 keywords for the whole video on the same scripts.

9. 2 week plan

Days	Task	Depends on	How I test it	Deliverable
1 to 2	Define the scene JSON format and study how to split a script into scenes	Voice timings from the pipeline	Split my ocean script by hand	Scene JSON format with one example
3 to 5	Build the scene splitter with an LLM prompt, using sentence timings	Day 1 to 2 format, Gemini API	Run it on 5 English and 5 Arabic scripts	Working scene splitter
6 to 8	Generate an English search phrase for each scene, including Arabic scripts, and search Pixabay	Scene splitter	Check if each clip matches its sentence	Clips chosen per scene
9 to 10	Compare with MoneyPrinterTurbo on the same scripts and prepare the demo	All of the above	Match rate: my module vs 5 keywords	Short results report and demo video

The plan starts with the format, because every other part depends on it. By the end of the two weeks I want to show the same script made two ways, once with MoneyPrinterTurbo's keywords and once with scene by scene matching, so the difference is clear.

10. Risks and open questions

- **Dialects are a problem.** Asking for Palestinian dialect gave me a dialect script, but I only tested the script, not the voice. My full Arabic video used formal Arabic, and the voice read it as formal Arabic. Can any TTS voice pronounce Palestinian or other dialects naturally? This is still an open question.
- **Outside services can stop working.** I tried to get a Pexels key, but Pexels had paused new API keys, even though it's the project's default footage source. The free Edge voice was also blocked. If a provider stops, the product needs a backup.
- **Font licences.** My Arabic fix used Tahoma, which is a Microsoft font, so we may not be allowed to ship it. A real product would need a free Arabic font, like Noto Sans Arabic.